

dtcu_basic: “DTCU 단독 정상동작” 검증

1.1 목표(왜 하는가)

dtcu_basic은 딱 하나를 확실히 보려는 테스트입니다.

1. 핸드셰이크가 되는가?
 - 커널이 DTENSOR_START로 DTCU 작업을 kick-off하고
 - DTENSOR_POLL로 done을 확인해서
 - 정상적으로 빠져나오는가(= hang 없이 종료되는가)
2. 메모리 경유 I/O가 맞는가?
 - descriptor(= dtensor_desc) / A,B operand / D output이
 - “코어 Register File”이 아니라 메모리 주소 기반으로 움직이고
 - 결과 D가 host가 계산한 reference와 일치하는가

즉, “DTCU 기능/연결이 살아있다”를 가장 작은 케이스로 확인하는 게 목적입니다.

1.2 구성 요소(파일이 뭘 하는가)

보통 dtcu_basic은 3개 레이어로 나뉩니다.

(A) Host side (tests/regression/dtcu_basic/main.cpp)

Host(= x86에서 도는 dtcu_basic executable)가 하는 일은 다음 순서입니다.

1. 입력 행렬 A,B를 CPU에서 생성
2. CPU에서 reference 결과(ref D)를 계산
3. Vortex runtime으로 device 열기 (vx_dev_open)
4. device memory 할당

- `vx_mem_alloc`로 A_buf, B_buf, D_buf, desc_buf를 잡고
 - `vx_mem_address`로 각 버퍼의 device virtual address를 얻음
5. descriptor를 host에서 구성해서 desc_buf에 복사
- `ptrA/ptrB/ptrD, ldmA/ldmB/ldmD, fmt_s/fmt_d, flags` 설정
 - 여기서 중요한 게 ****DTCU가 descriptor를 읽는 방식(특히 ldm/format/flags)****과 host가 맞춰야 한다는 점입니다.
6. 커널 인자(`kernel_arg_t`)를 upload해서 args_buffer 생성 (`vx_upload_bytes`)
7. 커널 바이너리(`kernel.vxbin`) 업로드 (`vx_upload_kernel_file`) → `krnl_buffer`
8. 실행 (`vx_start(device, krnl_buffer, args_buffer)`)
9. D_buf에서 결과를 다시 읽어오고(`vx_copy_from_dev`), reference와 비교

즉 **Host**는 “메모리 준비/실행/검증”만 하고, **DTCU**와 직접 통신하지 않습니다.
DTCU 통신은 “커널이 custom instruction을 통해” 수행합니다.

(B) Kernel side (tests/regression/dtcu_basic/kernel.cpp)

Kernel(= RV32에서 도는 kernel.elf/kernel.vxbin)이 하는 일은 매우 단순합니다.

1. (보통 1 thread만) `dtensor_start(desc_addr)` 실행
 - 이게 `DTENSOR_START` custom instruction을 emit합니다.
2. done이 될 때까지 `dtensor_poll()`을 루프
 - `DTENSOR_POLL` custom instruction을 반복
3. done이면 그냥 return (커널 종료)

여기서 핵심은:

커널은 **A/B/D**를 직접 **load/store** 하지 않습니다.

A/B/D는 “DTCU가 descriptor 주소를 따라가 RAM에서 직접 read/write”합니다.

(C) SimX microarchitecture side

이제부터가 진짜 중요한 상호작용입니다.

1) DTENSOR_START의 흐름

- Kernel이 DTENSOR_START를 실행
- SimX Emulator::decode()가 이 instruction을 **FUType::TCU(혹은 TcuType)**로 decode
 - 여기서 -DEXT_TCU_ENABLE이 없으면 decode가 이 경로를 모르고 abort하게 됩니다. (당신이 실제로 경험하셨던 abort 원인)
- execute stage에서 해당 op를 처리할 때:
 - core_id == 0 같은 조건을 걸어 “대표 core만” DTCU를 kick하도록 했을 가능성이 큼니다.
 - cluster->dtensor()->start(desc_addr) 호출

2) DTCU가 descriptor를 읽는 위치

- DTensorCore::start(desc_addr)는 내부 state를 “busy”로 만들고
- 다음 tick에서 ram->read(desc_, desc_addr, sizeof(desc_)) 같은 방식으로 descriptor를 읽습니다.

여기서 “cache_sim 구조가 독특해서 실제론 direct RAM read”라고 하신 부분이 바로 이 지점입니다.

즉, L2를 경유한 latency model은 “대표로만” 반영될 수 있지만, 기능적으로는 RAM에서 descriptor를 직접 읽습니다.

3) operand fetch / output store

- descriptor의 ptrA/ptrB/ptrC/ptrD, ldmA/B/C/D, fmt_s/fmt_d, flags를 보고
- DTCU가 A,B를 RAM에서 읽고(혹은 읽는 것처럼 모델링하고)
- 연산 후 D를 RAM에 씁니다(혹은 쓰는 것처럼 모델링)

1.3 L2 cache sim / mem req랑 “어떻게” 상호작용하나

이게 사용자님이 가장 궁금하신 포인트일 텐데, `dtcu_basic`에서의 상호작용은 두 레벨입니다.

레벨 1: 기능적 데이터(진짜 값) 관점

- `descriptor/operand/output`은 **RAM** 접근 **API**로 직접 읽고 쓰기가 발생합니다.
- 즉, 값의 **correctness**는 RAM read/write 로직이 결정합니다.

레벨 2: “통계/타이밍 모델” 관점

- DTCU는 cluster에 붙어 있고, L2 cache sim의 “추가 포트(마지막 port)”에 bind되어 있으니,
- DTCU는 상태머신 중 특정 단계에서 `mem_req_out`로 **MemReq**를 내보내고
- L2가 그 req를 받아 service한 뒤 `mem_rsp_in`으로 **MemRsp**를 넣어줍니다.

하지만 지금 구현 설명(그리고 당신이 말씀하신 “직접 메모리에서 가져오는 형식”)을 보면,

- 실제 `operand` 전체를 다 **MemReq**로 모델링하는 게 아니라
- “대표 요청 몇 개”만 날려서 L2 counter/latency를 대략 반영할 가능성이 큼니다.

그래서 `dtcu_basic`은 “L2 통계 정확도”를 주 검증 대상으로 보지 않고,

- 핸드셰이크 + 최종 결과값을 봅니다.
-

2) dtcu_compare: “DTCU vs in-core TCU 비교(값 + 통계)” 검증

2.1 목표(왜 하는가)

`dtcu_compare`는 `dtcu_basic`보다 목적이 더 큼니다.

1. 동일한 수학 연산을 두 경로로 수행
 - 경로 A: DTCU (`DTENSOR_START/POLL`)
 - 경로 B: 기존 in-core TCU (`wmma_context` 기반, 기존 `sgemm_tcu`/유사 test 재사용)
2. 결과값이 동일한지 확인
 - `D_dtcu == D_tcu == CPU_ref`
3. 통계적 차이를 뽑아내기
 - `cycles`, `instrs`, (가능하면) L2 read/write/miss 등
 - 즉, “DTCU가 core pipeline에서 instruction overhead를 얼마나 줄였는지” 같은 방향의 비교를 가능하게 합니다.

이 테스트는 “정상동작”을 넘어서, ‘비교 가능성(**comparability**)’을 만들어 주는 **regression**입니다.

2.2 구성 요소(파일이 뭘 하는가)

(A) Host side (`dtcu_compare/main.cpp`)

Host가 하는 일은 다음과 같이 “두 번 실행”입니다.

1. A,B 생성 + CPU reference 계산
2. device memory 할당 (A,B, D, desc 등)
3. **mode=0**으로 커널 실행

- DTCU 경로 수행
 - 결과를 `D_dtcu`로 readback
4. **mode=1**으로 커널 다시 실행
- in-core TCU(wmma) 경로 수행
 - 결과를 `D_tcu`로 readback
5. `D_dtcu`, `D_tcu`, `ref` 삼자 비교
6. (옵션) perf counter/통계 출력

여기서 중요한 건, 같은 커널 바이너리를 **mode**로 분기하는 방식이 **regression** 재사용성이 좋다는 점입니다.

(기존 regression들이 “한 app 안에서 여러 모드”로 결과 비교하는 패턴을 자주 씁니다.)

(B) Kernel side (dtcu_compare/kernel.cpp)

Kernel은 `arg->mode`에 따라 갈라집니다.

- `mode == 0`: dtcu_basic과 동일하게
 - `dtensor_start(desc_addr)`
 - `while(!dtensor_poll()) {}`
- `mode == 1`: in-core TCU 경로
 - 기존 `sgemm_tcu`의 `wmma_context<...>` 사용 방식(= 템플릿/fragment/load/mma/store)을 그대로 재활용
 - 이 때 **B**의 **layout(row_major / col_major)**, **ldm** 같은 게 DTCU와 정확히 맞아야 “값 비교”가 의미가 있습니다.
 - 즉, dtcu_compare는 “비교”가 목표라서, 이 부분이 dtcu_basic보다 훨씬 엄격합니다.

2.3 L2 cache sim / mem req 상호작용(비교 관점)

dtcu_compare에서 L2와의 관계는 “값”보다 “통계”에서 중요해집니다.

DTCU 경로(mode=0)

- DTCU는 cluster 레벨 모듈이라서, 본질적으로 “core가 A/B를 레지스터로 들고 들어가는 것”이 아닙니다.
- 그래서 이상적으로는:
 - core instruction 수는 매우 적음 (start/poll)
 - 메모리 트래픽은 DTCU가 만들어냄

하지만 당신 구현에서 “직접 RAM read/write”가 강하게 들어가 있으면,

- L2 mem req는 representative로만 찍힐 수 있어
- 통계가 과소/과대 추정될 수 있습니다.

그럼에도 dtcu_compare의 가치는 남습니다:

- instruction count/cycle count는 start/poll overhead 측정에 도움
- L2 counter는 “현재 모델의 수준에서” 상대 비교

in-core TCU 경로(mode=1)

- in-core TCU는 보통:
 - operand를 load하는 경로가 core pipeline/L1/L2를 적극적으로 사용하고,
 - 더 많은 core instruction이 발생합니다(주소계산/로드/스케줄링 등).
- 그래서 통계적으로:
 - instrs는 더 크게,
 - L2 read/write도 더 크게 나오는 게 자연스러운 방향입니다.

즉 dtcu_compare는 ***DTCU가 core pipeline 부담을 줄이고, 메모리 접근 주체를 core→cluster로 이동시킨다***는 방향성을 정량화할 수 있게 해줍니다.

3) dtcu_basic vs dtcu_compare: 차이 요약(의도/상호작용 관점)

구분	dtcu_basic	dtcu_compare
1차 목표	“DTCU가 도는가” (handshake + correctness)	“DTCU vs in-core TCU 비교” (correctness + 통계)
실행 횟수	1회 (DTCU만)	2회 (DTCU + in-core)
커널 동작	start/poll만	start/poll 또는 wmma 수행
메모리 상호작용의 의미	correctness 중심 (descriptor/operand/output)	correctness + (가능하면) L2/perf 차이
L2 cache sim 활용	있어도 부수적(모델 단순화 가능)	비교 지표로 중요해짐 (단, 모델 정확도 한계 존재)